

CONTENTS

Grafos: representación, BFS, DFS, componentes conexas, bipartición

Guía de estudio con tolerancia cero — Módulo 3, Programación 3 (FING Udelar) — Capítulo 3 de Kleinberg & Tardos

0. Por qué existe este tema

1. Representación de grafos: lista de adyacencia vs. matriz de adyacencia

Matriz de adyacencia

Lista de adyacencia

2. BFS (Breadth-First Search)

2.1 El algoritmo genérico de exploración (KT 3.2)

2.2 Pseudocódigo (notación exacta del curso)

2.3 Invariante de capas y camino más corto

2.4 Complejidad: $O(n+m)$

3. DFS (Depth-First Search)

3.1 Pseudocódigo iterativo (notación exacta del curso — con pila explícita)

3.2 Propiedad de “no cruce” y clasificación de aristas

3.3 Detección de ciclos vía DFS (Ejercicio 4 / KT 3.2)

3.4 Complejidad: $O(n+m)$

4. Componentes conexas (grafos no dirigidos)

5. Bipartición (2-coloreo)

Teorema: G es bipartito $\Leftrightarrow G$ no tiene ciclos de longitud impar

Algoritmo (2-coloreo vía BFS, capa por capa)

6. Grafos dirigidos: fuertemente conexo y orden topológico

6.1 Fuertemente conexo (SCC)

6.2 Orden topológico

7. Trampas típicas de examen

Ejercicios de repaso rápido (con respuesta)

Resumen ultra-rápido (2 minutos)

Grafos: representación, BFS, DFS, componentes conexas, bipartición

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 3, PROGRAMACIÓN 3 (FING UDELAR) — CAPÍTULO 3 DE KLEINBERG & TARDOS

0. Por qué existe este tema

Casi todos los algoritmos “grandes” del resto del curso (Greedy sobre grafos, Programación Dinámica sobre DAGs, Flujo Máximo) asumen como caja negra que ya sabés **recorrer un grafo completo, sin repetir nodos y sin dejar ninguno sin visitar, en tiempo lineal**. Este módulo es esa caja negra: dos formas concretas (BFS y DFS) de instanciar un esquema genérico de “descubrir nodos alcanzables”, y un puñado de propiedades sobre esas dos formas que se reutilizan una y otra vez — capas de distancia, aristas de árbol vs. aristas de retorno, invariantes de coloreo.

La razón por la que este módulo pesa tanto en los parciales es doble:

1. **Es la base de razonamiento de casi todo lo que sigue.** El patrón “reducir un problema nuevo a BFS/DFS/bipartición/orden topológico ya conocido” (ver Ejercicios 8 y 12 del Práctico 3) es, según la resolución del práctico, el patrón más transferible del curso.
2. **La demostración de complejidad $O(n+m)$** que aparece acá (cada nodo se procesa $O(1)$ veces, cada arista se examina $O(1)$ veces) es el esqueleto de casi todas las demostraciones de complejidad del resto de la materia.

Regla de oro que se repite en TODO el módulo: en un grafo $G=(V,E)$ con $n=|V|$ nodos y $m=|E|$ aristas, **todo** algoritmo de recorrida bien hecho (BFS, DFS, componentes conexas, bipartición, orden topológico) corre en **$O(n+m)$** , porque cada nodo se marca/procesa una cantidad constante de veces y cada arista se examina una cantidad constante de veces (a lo sumo 2 en no dirigido, a lo sumo 1 en dirigido). Si en un examen te da una complejidad distinta, o programaste mal, o el enunciado te pide algo adicional (ej. reconstruir un camino, contar caminos) que hay que sumar aparte.

1. Representación de grafos: lista de adyacencia vs. matriz de adyacencia

Notación estándar del curso (estilo KT): n_v denota el **grado** de un nodo v = cantidad de aristas incidentes a v . La identidad clave, usada en casi todas las demostraciones de complejidad de este módulo:

$$\sum_{v \in V} n_v = 2m \text{ (en grafos no dirigidos)}$$

porque cada arista se cuenta exactamente dos veces, una desde cada extremo.

MATRIZ DE ADYACENCIA

Matriz A de $n \times n$ donde $A[u,v] = 1$ si el grafo contiene la arista (u,v) , y 0 si no.

- **Espacio:** $O(n^2)$ — siempre, sin importar cuántas aristas tenga realmente el grafo.
- **Consultar si $(u,v) \in E$:** $O(1)$ — acceso directo a $A[u,v]$.
- **Listar los vecinos de un nodo v :** $O(n)$ — hay que recorrer toda la fila/columna de v , aunque tenga pocos vecinos.

LISTA DE ADYACENCIA

Un registro (record) por cada nodo v , conteniendo la lista de nodos a los que v tiene una arista. Array `Adj` donde `Adj[v]` es esa lista.

- **Espacio:** $O(n+m)$ — n registros (uno por nodo, aunque esté vacío) más m entradas de arista en total ($2m$ si no dirigido, porque cada arista aparece en la lista de sus dos extremos).
- **Consultar si $(u,v) \in E$:** $O(\text{grado}(u))$ en el peor caso — hay que recorrer la lista de u .
- **Listar los vecinos de un nodo v :** $O(\text{grado}(v))$ — tiempo **proporcional a la cantidad real de vecinos**, no a n .

La razón por la que BFS/DFS se implementan casi siempre sobre **lista de adyacencia** y no sobre matriz: como $m \leq n^2$, el espacio $O(n+m)$ nunca es peor que $O(n^2)$, y en la práctica casi todos los grafos de examen son **dispersos** (m mucho menor que n^2) — ahí la lista gana en espacio Y en tiempo de recorrida, porque recorrer “todos los vecinos de v ” cuesta $\text{grado}(v)$ y no n . La matriz solo conviene cuando el grafo es denso (m cercano a n^2) o cuando se necesita consultar aristas puntuales en $O(1)$ con muchísima frecuencia.

	Matriz de adyacencia	Lista de adyacencia
Espacio	$O(n^2)$	$O(n+m)$
$\checkmark(u,v) \in E?$	$O(1)$	$O(\text{grado}(u))$
Listar vecinos de v	$O(n)$	$O(\text{grado}(v))$
Conviene cuando...	grafo denso ($m \approx n^2$), muchas consultas puntuales de arista	grafo disperso ($m \ll n^2$) — el caso típico de BFS/DFS

Error frecuente de tolerancia cero: decir que la lista de adyacencia tiene espacio $O(m)$ a secas. Es **$O(n+m)$** : aunque un grafo tenga pocas aristas (incluso $m=0$), sigue habiendo que reservar un registro por cada uno de los n nodos (aunque esté vacío) para poder indexar `Adj[v]` en $O(1)$. Ese “ $+n$ ” no se puede omitir.

2. BFS (Breadth-First Search)

2.1 EL ALGORITMO GENÉRICO DE EXPLORACIÓN (KT 3.2)

BFS y DFS son ambos **casos particulares** de un esquema más general: mantener un conjunto S de nodos “descubiertos”, empezar con $S=\{s\}$, y mientras exista una arista (u,v) con $u \in S$ y $v \notin S$, agregar v a S . Este esquema genérico, sin especificar en qué orden se procesan los nodos de S , ya alcanza para calcular la **compo-**

nente conexa de s (todo lo alcanzable desde s) — pero no da información de distancias ni estructura de árbol. BFS y DFS agregan esa información extra, cada uno especializando el orden de procesamiento de una forma distinta.

2.2 PSEUDOCÓDIGO (NOTACIÓN EXACTA DEL CURSO)

```

Algorithm BFS(s)
  descubierto[s] = true
  Inicializar cola conteniendo solo s
  nivel[s] = 0
  while cola no está vacía do
    u = Retirar un vértice de cola      // FIFO: el primero en entrar es el primero en salir
    /* Acá se explora u */
    foreach v adyacente a u do
      if not descubierto[v] then
        descubierto[v] = true
        padre[v] = u
        nivel[v] = nivel[u] + 1
        Agregar v a cola
      end
    end
  end
end
end

```

Para recorrer **todo** el grafo (no solo la componente de un s fijo), se envuelve en un bucle exterior:

```

Algorithm ComponentesConexasBFS
  Hacer descubierto[u] = false para todo  $u \in V$ 
  for  $s = 1$  to  $n$  do
    if not descubierto[s] then
      BFS(s)      // cada llamada nueva a BFS descubre una componente conexa completa
    end
  end
end
end

```

El detalle que separa BFS de la exploración genérica es **una sola línea**: la estructura de datos donde se guardan los nodos “descubiertos pero no explorados todavía” es una **cola FIFO**. Eso es lo que fuerza a que se procesen en el orden en que fueron descubiertos, y por lo tanto en orden creciente de distancia a s — de ahí sale toda la teoría de capas de la sección 2.3.

2.3 INVARIANTE DE CAPAS Y CAMINO MÁS CORTO

Definí L_i = conjunto de nodos a distancia exactamente i de s (siguiendo el orden en que BFS los descubre por niveles).

Propiedad 3.3 (KT). Para todo $j \geq 1$, la capa L_j generada por BFS consiste exactamente en los nodos a distancia j de s . Existe un camino de s a t **si y solo si** t aparece en alguna capa.

Propiedad 3.4 (KT) — “no salto de capas”. Sea T el árbol BFS, x e y nodos en las capas L_i , L_j respectivamente, y sea (x,y) una arista de G (esté o no en T). Entonces **i y j difieren en a lo sumo 1**.

Idea de la demostración: si difirieran en 2 o más (por ejemplo $j > i + 1$), entonces cuando BFS estaba procesando la capa L_i y llegó a la arista (x, y) , habría descubierto a y en ese momento — es decir, y estaría en la capa L_{i+1} a más tardar, contradiciendo que esté en L_j con $j > i + 1$.

Consecuencia directa (por qué BFS da caminos más cortos): como ninguna arista de G puede “saltar” más de una capa, cualquier camino de s a t debe atravesar las capas $L_0, L_1, \dots, L_{\text{nivel}(t)}$ en orden, una por una, sin poder saltarse ninguna. Por lo tanto **nivel(t) es exactamente la distancia (en cantidad de aristas) más corta de s a t** , y el camino que se obtiene siguiendo los punteros `padre[·]` desde t hasta s es un camino más corto. Esto vale **solo para grafos no pesados** (o con todas las aristas de igual peso) — ver la trampa de la sección 7.

2.4 COMPLEJIDAD: $O(N+M)$

- Cada nodo se marca `descubierto` exactamente una vez $\rightarrow$ se encola y desencola $O(1)$ veces cada uno $\rightarrow O(n)$ en el manejo de la cola.
- Al desencolar u , se recorre la lista de adyacencia de u completa: `foreach v adyacente a u`. Sumado sobre todos los nodos, esto es $O(\sum_u \text{grado}(u)) = O(2m) = O(m)$.
- Total: **$O(n+m)$** , asumiendo lista de adyacencia (con matriz sería $O(n^2)$), porque cada `foreach v adyacente a u` costaría $O(n)$ en vez de $O(\text{grado}(u))$.

Ejemplo 2.1 — BFS paso a paso sobre un grafo concreto de 8 nodos, verificado con Python.

Grafo A (no dirigido, conexo, 8 nodos, 9 aristas): A-B, A-C, B-D, B-E, C-F, D-G, E-G, F-H, G-H.

Listas de adyacencia (orden alfabético, así se recorren los vecinos): A:[B,C] · B:[A,D,E] · C:[A,F] · D:[B,G] · E:[B,G] · F:[C,H] · G:[D,E,H] · H:[F,G].

Traza de BFS desde A (estado de la cola después de cada operación):

```
Inicio: cola=[A], nivel(A)=0
Desencolo A. Vecinos: B, C (ninguno descubierto)
  descubro B desde A (nivel 1), cola=[B]
  descubro C desde A (nivel 1), cola=[B, C]
Desencolo B. Vecinos: A(ya), D, E
  descubro D desde B (nivel 2), cola=[C, D]
  descubro E desde B (nivel 2), cola=[C, D, E]
Desencolo C. Vecinos: A(ya), F
  descubro F desde C (nivel 2), cola=[D, E, F]
Desencolo D. Vecinos: B(ya), G
  descubro G desde D (nivel 3), cola=[E, F, G]
Desencolo E. Vecinos: B(ya), G(ya) -> nada nuevo, cola=[F, G]
Desencolo F. Vecinos: C(ya), H
  descubro H desde F (nivel 3), cola=[G, H]
Desencolo G. Vecinos: D(ya), E(ya), H(ya) -> nada nuevo, cola=[H]
Desencolo H. Vecinos: F(ya), G(ya) -> nada nuevo, cola=[]
```

Capas resultantes: $L_0=\{A\}$, $L_1=\{B,C\}$, $L_2=\{D,E,F\}$, $L_3=\{G,H\}$.

Árbol BFS (padre[·]): $B \leftarrow A$, $C \leftarrow A$, $D \leftarrow B$, $E \leftarrow B$, $F \leftarrow C$, $G \leftarrow D$, $H \leftarrow F$.

Verificación con Python (script ejecutado, salida real):

```

Orden de procesamiento (desencolado): ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
Niveles: {'A': 0, 'B': 1, 'C': 1, 'D': 2, 'E': 2, 'F': 2, 'G': 3, 'H': 3}
Padres: {'A': None, 'B': 'A', 'C': 'A', 'D': 'B', 'E': 'B', 'F': 'C', 'G': 'D', 'H': 'F'}
Capas: {0: ['A'], 1: ['B', 'C'], 2: ['D', 'E', 'F'], 3: ['G', 'H']}

```

Coincide exactamente con la traza a mano. El script además verificó automáticamente, recorriendo las 9 aristas, que **ninguna conecta capas que difieran en más de 1** (Propiedad 3.4) — por ejemplo la arista G-H conecta dos nodos de la misma capa L_3 (diferencia 0), y D-G conecta L_2 - L_3 (diferencia 1); ninguna arista conecta, por ejemplo, L_1 con L_3 .

Camino más corto de A a H: siguiendo padre[.] desde H hacia atrás: $H \leftarrow F \leftarrow C \leftarrow A$, es decir A-C-F-H, de longitud $3 = \text{nivel}(H) \checkmark$.

3. DFS (Depth-First Search)

3.1 PSEUDOCÓDIGO ITERATIVO (NOTACIÓN EXACTA DEL CURSO — CON PILA EXPLÍCITA)

Esta es la forma en que el curso lo define (no la versión recursiva “de libro de texto”), y tiene un detalle no trivial: **un mismo nodo puede quedar apilado varias veces**, y solo la primera vez que se desapila sin estar aún explorado se procesa de verdad.

```

Algorithm DFS(s)
  Inicializar pila conteniendo solo s
  while pila no está vacía do
    u = Retirar un vértice de pila      // LIFO
    if not explorado[u] then
      explorado[u] = true
      /* Acá se explora u */
      foreach v adyacente a u do
        Agregar v a pila
        if not explorado[v] then padre[v] = u
      end
    end
  end
end

```

```

Algorithm ComponentesConexasDFS
  Hacer explorado[u] = false para todo u ∈ V
  for s = 1 to n do
    if not explorado[s] then
      DFS(s)
    end
  end
end

```


Por qué se apilan duplicados a propósito: cuando se procesa u , se apilan *todos* sus vecinos v de una vez — incluso si alguno ya fue apilado antes por otro nodo. Recién cuando ese v efectivamente se desapila y su chequeo `not explorado[v]` sigue dando true, se procesa; si para entonces ya fue explorado por otro camino, esa copia se descarta sin hacer nada. Cada nodo puede aparecer en la pila hasta $\text{grado}(v)$ veces, pero se **procesa** exactamente una — esto es clave para la demostración de complejidad (sección 3.4) y para no confundirse contando “cuántas veces se apila” con “cuántas veces se explora”.

Versión recursiva equivalente (más cómoda para clasificar aristas, sección 3.2):

```
DFS(u):
    color[u] = gris           // "en la pila de recursión"
    foreach v adyacente a u do
        if color[v] == blanco then
            padre[v] = u
            DFS(v)             // arista de árbol
        else if color[v] == gris and v ≠ padre[u] then
            // arista hacia atrás (back edge) -> hay un ciclo
        color[u] = negro       // "terminado"
```

3.2 PROPIEDAD DE “NO CRUCE” Y CLASIFICACIÓN DE ARISTAS

Propiedad 3.6 (KT). Para una llamada recursiva $\text{DFS}(u)$ dada, todos los nodos marcados como “explorados” entre la invocación y el final de esa llamada son **descendientes de u** en el árbol T .

Propiedad 3.7 (KT) — la propiedad clave del módulo. Sea T un árbol DFS, sean x e y nodos de T , y sea (x,y) una arista de G que **no** es arista de T . Entonces **x es ancestro de y , o y es ancestro de x** en T .

En otras palabras: en un grafo **no dirigido**, DFS **nunca genera aristas “cruzadas”** entre dos ramas distintas del árbol (a diferencia de BFS, que sí puede tener aristas entre nodos de la misma capa que no son ancestro/descendiente uno del otro). Toda arista de G que no es de árbol conecta a un nodo con uno de sus propios ancestros — es una **arista hacia atrás (back edge)**.

Por qué: supongamos que DFS visita primero a x (WLOG). Por la Propiedad 3.6, mientras x sigue en la pila de recursión (color gris), la arista (x,y) va a ser examinada en algún momento — si y todavía es blanco, se vuelve descendiente directo de x (arista de árbol, contradice la hipótesis); si y ya es gris, entonces y es un ancestro de x en la recursión actual (está en el camino desde la raíz hasta x); si y ya es negro, eso es imposible: y no puede haber terminado (color negro) antes de que x termine de examinar la arista (x,y) , porque para que y termine, todos sus descendientes deben haber terminado — y como x todavía está activo cuando se examina (x,y) , x no puede ser descendiente de un y ya terminado sin contradicción de tiempos.

Esta propiedad es la que distingue a un grafo **no dirigido**: en dirigido sí pueden aparecer *forward edges* (hacia un descendiente ya visitado por otro camino) y *cross edges* (entre ramas sin relación de ancestro) — por eso la detección de ciclos y el orden topológico en dirigidos necesitan la clasificación completa de 4 tipos de arista (árbol, back, forward, cross), mientras que en no dirigido **alcanza con mirar solo si hay una back edge** para saber si hay ciclo.

3.3 DETECCIÓN DE CICLOS VÍA DFS (EJERCICIO 4 / KT 3.2)

Algoritmo: correr DFS manteniendo `padre[v]` y un marcador `enPila[v]` (“en la pila de recursión actual”). Al explorar la arista (u,v) :

- Si v no visitado: recursión normal.
- Si v visitado, `enPila[v]=true` y $v \neq \text{padre}[u]$: se encontró un **ciclo** — es una back edge. Se reconstruye siguiendo `padre[·]` desde u hacia atrás hasta v , y agregando la arista (u,v) al final.
- Si v visitado pero `enPila[v]=false`: no hay ciclo por ahí (es de una rama ya terminada).

La corrección se apoya exactamente en la Propiedad 3.7: **en un grafo no dirigido, hay un ciclo si y solo si el DFS encuentra una back edge** (una arista a un ancestro todavía activo en la recursión).

Ejemplo 3.1 — DFS con clasificación de aristas sobre el mismo grafo A, verificado con Python (detecta que el grafo NO es bipartito — ver sección 5).

DFS recursivo desde A (orden alfabético de vecinos), salida real del script:

```
Orden de exploración: A, B, D, G, E, H, F, C
Aristas de árbol: (A,B) (B,D) (D,G) (G,E) (G,H) (H,F) (F,C)
Aristas hacia atrás (back edges): (E,B) (C,A)
```

Árbol DFS resultante (con indentación = profundidad):

```

A
├── B
│   ├── D
│   │   ├── G
│   │   │   ├── E ..... back edge (E,B): E es descendiente de B, cierra el ciclo B-D-G-E-B
│   │   │   ├── H
│   │   │       ├── F
│   │   │           ├── C ..... back edge (C,A): C es descendiente de A, cierra el ciclo A-B-D-G-H-F-C-A
```

Ciclos detectados: B-D-G-E-B (longitud 4, par) y A-B-D-G-H-F-C-A (longitud 7, impar). El segundo ciclo, de longitud impar, es lo que hace que el grafo A **no sea bipartito** (Teorema de la sección 5) — confirmado independientemente por el chequeo de 2-coloreo en el mismo script, que reporta conflicto en la arista G-H.

Nótese que ninguna arista del grafo conecta dos ramas “hermanas” sin relación de ancestro (no hay cross edges) — consistente con la Propiedad 3.7.

3.4 COMPLEJIDAD: $O(N+M)$

Con la versión iterativa del curso: el paso principal es agregar/quitar nodos de la pila, $O(1)$ cada operación. Para acotar el tiempo total alcanza con contar cuántos nodos se **agregan** a la pila en total: un nodo v se agrega cada vez que uno de sus n_v vecinos es explorado, así que el total de inserciones es a lo sumo $\sum_{v \in V} n_v = 2m$ (Propiedad 3.9). Por lo tanto el manejo de la pila es $O(m)$, más $O(n)$ adicional para inicializar los arreglos `explorado[·]` y recorrer el bucle exterior de componentes conexas. Total: $O(n+m)$.

⚠ **Detalle de precisión que conviene marcar en el examen:** en la versión iterativa del curso, un nodo puede aparecer en la pila hasta grado(v) veces (no una sola vez como en la versión recursiva “de libro”), así que el tamaño máximo de la pila en un instante dado puede ser $O(m)$, no $O(n)$. Esto NO cambia la complejidad total del algoritmo (sigue siendo $O(n+m)$ en tiempo), pero si el examen pregunta específicamente por el **espacio auxiliar** de la pila en esta implementación iterativa, la respuesta correcta es $O(m)$ en el peor caso, no $O(n)$ — a diferencia de la versión recursiva (o de BFS con cola), donde cada nodo ocupa la estructura una sola vez y el espacio auxiliar es $O(n)$.

4. Componentes conexas (grafos no dirigidos)

Definición. Dado un nodo s , su componente conexa es el conjunto R de todos los nodos alcanzables desde s por algún camino. **Propiedad 3.8 (KT):** para cualquier par de nodos, sus componentes conexas son **idénticas o disjuntas** — nunca se solapan parcialmente (si u y v están conectados, tienen exactamente la misma componente; si no, sus componentes no comparten ningún nodo).

Por qué BFS/DFS la resuelven en una sola pasada: el conjunto de nodos que BFS (o DFS) marca como `descubierto / explorado` a partir de s es, por construcción, exactamente el conjunto de nodos alcanzables desde s — ni más (nunca se descubre un nodo sin un camino real desde s) ni menos (todo nodo alcanzable termina siendo descubierto, porque mientras exista un nodo descubierto con un vecino sin descubrir, el algoritmo sigue explorando). Es la misma garantía que el algoritmo genérico de exploración de la sección 2.1, solo que BFS/DFS además dejan una estructura de árbol.

Algoritmo para calcular TODAS las componentes de un grafo (no solo la de un nodo dado): el bucle exterior `for s=1 to n: if not descubierto[s] then BFS(s)` (secciones 2.2/3.1). Cada vez que se entra al `if`, s es un nodo que ninguna llamada anterior alcanzó — por lo tanto **empieza una componente nueva**, y esa llamada a BFS/DFS la completa entera antes de seguir. El número total de llamadas a BFS/DFS a lo largo de todo el bucle es exactamente la cantidad de componentes conexas del grafo.

Ejemplo 4.1 — Componentes conexas de un grafo desconectado de 11 nodos, verificado con Python.

Grafo B (no dirigido, 11 nodos, DOS componentes):

- Componente 1 (ciclo de 6): 1-2, 2-3, 3-4, 4-5, 5-6, 6-1.
- Componente 2 (triángulo + cola): 7-8, 8-9, 9-7, 9-10, 10-11.

Corriendo el bucle exterior: `s=1` no está descubierto $\rightarrow$ BFS(1) descubre $\{1,2,3,4,5,6\}$ entera (es un ciclo, todos alcanzables entre sí). Se sigue el `for`: `s=2..6` ya están descubiertos, se saltean. `s=7` no está descubierto $\rightarrow$ BFS(7) descubre $\{7,8,9,10,11\}$. `s=8..11` ya descubiertos, se saltean. Fin: 2 llamadas a BFS = 2 componentes.

Salida real del script Python (BFS con cola, recorriendo `for s in adj` en orden 1..11):

Componentes conexas (grafo B, 11 nodos):

[1, 2, 3, 4, 5, 6]

[7, 8, 9, 10, 11]

Coincide exactamente con lo esperado por construcción del grafo.

Error frecuente de tolerancia cero — grafos desconectados: correr BFS/DFS **solo desde un nodo** y asumir que se recorrió “todo el grafo”. Si el grafo no es conexo, eso solo da la componente de ese nodo — el resto queda sin visitar y sin marcar. La solución correcta SIEMPRE es el bucle exterior `for s=1 to n: if not descubierto[s] then BFS(s)`, aunque el enunciado no diga explícitamente “el grafo puede ser desconectado”. En un examen, omitir ese bucle exterior (o escribir el algoritmo asumiendo implícitamente que el grafo es conexo sin decirlo) es un error grave de corrección, no un detalle de estilo.

5. Bipartición (2-coloreo)

Definición. Un grafo $G=(V,E)$ es **bipartito** si sus nodos se pueden partir en dos conjuntos X e Y tales que **toda** arista tiene un extremo en X y el otro en Y (ninguna arista queda “adentro” de X o de Y).

TEOREMA: G ES BIPARTITO $\Leftrightarrow G$ NO TIENE CICLOS DE LONGITUD IMPAR

($\Rightarrow$, si es bipartito entonces no hay ciclos impares). Por absurdo: supongamos G bipartito (partición $X \cup Y$) con un ciclo $v_1, v_2, \dots, v_k, v_1$ de longitud k impar. Como toda arista cruza entre las partes, los nodos del ciclo alternan estrictamente $X, Y, X, Y, \dots$: si $v_1 \in X$, entonces $v_i \in X$ si i es impar, $v_i \in Y$ si i es par. Para que la arista de cierre (v_k, v_1) sea válida (debe cruzar de parte), v_k debe estar en la parte opuesta a v_1 , es decir $v_k \in Y$ — lo que requiere k **par**. Pero asumimos k impar. Contradicción. ■

($\Leftarrow$, si no hay ciclos impares entonces es bipartito — constructiva, vía BFS de 2-coloreo). Se construye la partición con el algoritmo de la sección siguiente; si el algoritmo nunca detecta un conflicto de color, la coloración obtenida es, por construcción, una bipartición válida. Se prueba por contrapositiva: si el algoritmo detecta un conflicto en una arista (u,v) (mismo color en ambos extremos), esa arista más el camino en el árbol BFS entre u y v forman un ciclo, y ese ciclo tiene longitud impar (porque u y v terminaron con el mismo color de paridad, es decir están a la misma distancia par entre sí a lo largo del árbol, y la arista (u,v) cierra un camino de longitud par + 1 arista = impar).

ALGORITMO (2-COLOREO VÍA BFS, CAPA POR CAPA)

```
Bipartición(s):
  color[s] = 0
  BFS desde s, y al descubrir cada nodo v desde u:
    color[v] = 1 - color[u]      // color opuesto al de su padre
  al examinar cualquier arista (u,v) con ambos ya coloreados:
    si color[u] == color[v]: NO es bipartito (conflicto)
  repetir para cada componente conexas (son independientes entre sí)
```

Como BFS colorea por niveles, esto es exactamente $\text{color}[v] = \text{nivel}[v] \bmod 2$ — los nodos de nivel par van a X, los de nivel impar a Y. Esta es la razón profunda por la que funciona: dentro de una misma componente, la paridad de la distancia a la raíz determina el color, y por la Propiedad 3.4 (ninguna arista salta más de una capa), toda arista conecta capas iguales o adyacentes — si conecta capas adyacentes (paridad distinta), es consistente con bipartición; si conecta dos nodos de la **misma** capa (misma paridad), esa arista es la evidencia directa de un ciclo impar.

Ejemplo 5.1 — 2-coloreo sobre el grafo B (componente bipartita y componente no bipartita), verificado con Python.

Componente 1 {1,2,3,4,5,6} (ciclo de 6 — longitud par): BFS desde 1, $\text{color}[1]=0$. Nivel 1: 2 y 6, $\text{color}=1$. Nivel 2: 3 (vía 2) y 5 (vía 6), $\text{color}=0$. Nivel 3: 4 (vía 3 o 5), $\text{color}=1$. Ninguna arista conecta nodos del mismo color → **bipartito**, con $X=\{1,3,5\}$, $Y=\{2,4,6\}$.

Salida real del script: `bipartito=True, coloreo={1: 0, 2: 1, 6: 1, 3: 0, 5: 0, 4: 1}` ✓ coincide.

Componente 2 {7,8,9,10,11} (triángulo 7-8-9 + cola 9-10-11 — el triángulo es un ciclo de longitud 3, impar): BFS desde 7, $\text{color}[7]=0$. Nivel 1: 8 y 9, ambos $\text{color}=1$ (vecinos directos de 7). Al examinar la arista (8,9): 8 ya tiene $\text{color}=1$ y 9 ya tiene $\text{color}=1$ → **mismo color** → **conflicto** → **NO bipartito**.

Salida real del script: `bipartito=False, conflicto en arista (8, 9)` ✓ coincide — y en efecto 7-8-9-7 es un ciclo de longitud 3 (impar), consistente con el teorema.

Verificación cruzada con el grafo A (sección 3, Ejemplo 3.1): el ciclo impar A-B-D-G-H-F-C-A (longitud 7) detectado por DFS predice que el grafo A tampoco es bipartito. El chequeo de 2-coloreo del script lo confirma de forma independiente: `bipartito=False, conflicto en arista ('G', 'H')` — G y H terminan ambos en nivel 3 (mismo color), consistente con que la arista G-H “cierra” un ciclo de longitud impar.

Truco de verificación: si un ejercicio te pide “decidí si G es bipartito”, no hace falta encontrar el ciclo impar explícitamente para responder que NO lo es — alcanza con correr el 2-coloreo y reportar la primera arista en conflicto. Pero si además te piden **exhibir** el ciclo impar como certificado, reconstruilo igual que en la detección de ciclos (sección 3.3): seguí ``padre[.]`` desde ambos extremos de la arista conflictiva hasta el ancestro común (la raíz de BFS, si están en la misma componente), y esa concatenación de dos caminos + la arista conflictiva es el ciclo. Su longitud es siempre impar porque ambos extremos están en la misma paridad de nivel.

6. Grafos dirigidos: fuertemente conexo y orden topológico

(Este contenido sí aparece en este módulo: es parte del capítulo 3 de KT y de los Ejercicios 9-12 del Práctico 3, con dos de ellos — KT 3.3 y KT 3.12 — marcados como ejercicios de evaluación. Se incluye completo por eso.)

6.1 FUERTEMENTE CONEXO (SCC)

Definición. En un grafo dirigido, dos nodos u, v están en la misma **componente fuertemente conexa (SCC)** si u es alcanzable desde v y v es alcanzable desde u (hay camino dirigido en ambos sentidos). Una SCC es un conjunto **maximal** con esta propiedad.

Técnica clave: sea G^{rev} el grafo con todas las aristas invertidas. Para un nodo fijo A :

- DFS/BFS desde A en G da los nodos que A **alcanza**.
- DFS/BFS desde A en G^{rev} da los nodos que **alcanzan** a A (porque seguir una arista invertida (v, u) en G^{rev} equivale a seguir la arista original (u, v) “hacia atrás” en G).
- La **intersección** de ambos conjuntos es exactamente la SCC de A .

Para calcular TODAS las SCC de un grafo (algoritmo de Kosaraju):

1. DFS completo en G , registrando el orden de **finalización** de cada nodo (cuándo termina de explorarse, no cuándo se descubre).
2. Procesar los nodos en orden **decreciente** de finalización, haciendo DFS en G^{rev} desde cada nodo aún no visitado — cada árbol resultante de este segundo paso es exactamente una SCC completa.

6.2 ORDEN TOPOLÓGICO

Definición. Un orden topológico de un grafo dirigido es una secuencia $v_1, \dots, v_n$ tal que toda arista (v_i, v_j) cumple $i < j$.

Teorema. Existe un orden topológico de $G \Leftrightarrow G$ es un **DAG** (dirigido y acíclico — sin ciclos dirigidos).

($\Rightarrow$) si existiera un ciclo $v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_1$, cualquier orden topológico requeriría simultáneamente $\text{pos}(v_1) < \text{pos}(v_2) < \dots < \text{pos}(v_k) < \text{pos}(v_1)$ — imposible. ($\Leftarrow$) se prueba constructivamente con el algoritmo de Kahn (abajo): si G es acíclico, el algoritmo siempre logra ordenar los n nodos.

Algoritmo de Kahn — $O(n+m)$:

1. Calcular el grado de entrada de cada nodo (recorriendo todas las aristas una vez: $O(m)$).
2. Inicializar una cola con todos los nodos de grado de entrada 0 ($O(n)$).
3. Repetir: sacar un nodo u de la cola, agregarlo al orden, y decrementar el grado de entrada de cada vecino saliente de u ; si algún vecino llega a grado 0, encolarlo.
4. Cada arista se examina exactamente una vez en total (al decrementar el grado de su destino) $\rightarrow O(m)$.
Cada nodo se encola/procesa una vez $\rightarrow O(n)$. Total **$O(n+m)$** .

Corrección: en cada paso, el nodo elegido no tiene predecesores pendientes, así que agregarlo respeta el invariante de que todo lo ya ordenado precede correctamente a lo que falta. El algoritmo logra ordenar los n nodos **si y solo si** G es un DAG — si la cola se vacía y quedan nodos con grado de entrada > 0 entre sí, esos nodos forman un subgrafo donde todos tienen al menos un predecesor pendiente dentro del mismo subconjunto restante, lo cual fuerza la existencia de un ciclo (KT 3.3, Ejercicio 11 del práctico): siguiendo repetidamente cualquier arista entrante dentro de ese subconjunto restante, se termina revisitando un nodo — el segmento entre las dos ocurrencias es un ciclo real de G .

El patrón de reducción más importante del módulo (Ejercicio 12 / KT 3.12): muchos problemas de “consistencia de restricciones” (ej. cronología histórica con relaciones “antes” y “simultáneo”) se resuelven combinando **componentes conexas** (para agrupar lo que debe ir “junto”, vía relaciones simétricas de tipo “solapado”/“igual”) con **orden topológico** sobre el grafo de esas componentes (para las relaciones asimétricas de tipo “antes”/“diferente”). Si un ciclo aparece en el grafo dirigido resultante, los datos son inconsistentes; si dos elementos forzados a estar en la misma componente contradicen una restricción de orden estricto entre ellos, también es inconsistente de forma directa.

7. Trampas típicas de examen

Trampa 1 — confundir BFS con “camino más corto con pesos”. BFS solo garantiza caminos más cortos cuando **todas las aristas cuentan como “1 paso”** (grafo no pesado, o con pesos todos iguales). Si las aristas tienen pesos distintos, BFS puede dar un camino con menos aristas pero de peso total mayor que el óptimo — para ese problema hace falta Dijkstra (u otro algoritmo de caminos mínimos con pesos), que es un módulo posterior del curso, NO este. En un examen, si el enunciado menciona “costos”, “distancias en km”, “pesos” o cualquier magnitud que no sea “1 arista = 1 paso”, BFS solo no alcanza.

Trampa 2 — olvidar marcar visitados (o marcar tarde) y generar loops infinitos. Tanto en BFS como en DFS, un nodo se marca `descubierto / explorado` **en el momento en que se agrega a la estructura** (BFS: al encolar) o **al desencolarlo/desapilarlo por primera vez** (DFS iterativo del curso — ver sección 3.1). Si el marcado se hace en el momento equivocado (por ejemplo, en BFS, marcar como descubierto recién cuando se *procesa* el nodo en vez de cuando se *encola*), un mismo nodo puede encolarse múltiples veces desde distintos vecinos antes de ser marcado, inflando la complejidad de $O(n+m)$ a algo peor (en el peor caso, hasta $O(nm)$) — y en un grafo con ciclos, si el marcado falla del todo, el algoritmo puede efectivamente no terminar.

Trampa 3 — grafos desconectados: recorrer solo desde un nodo. Ver la caja de error de la sección 4. Si el enunciado no aclara que el grafo es conexo, **nunca asumirlo** — el bucle exterior `for s=1 to n: if not descubierto[s] then BFS(s)` es obligatorio para cualquier algoritmo que deba procesar “todo el grafo” (componentes conexas, bipartición completa, etc.).

Trampa 4 — pensar que el árbol BFS/DFS es único. Ni el árbol BFS ni el árbol DFS son únicos en general: si un nodo tiene más de un vecino candidato a “descubrirlo primero” (más de un vecino en el nivel anterior, en el caso de BFS; o el orden en que se listan/apilan los vecinos, en DFS), distintos órdenes de procesamiento de vecinos dan árboles distintos. Lo que ****s**** es invariante son las **capas** de BFS (Propiedad 3.3 — la distancia de cada nodo a la raíz no depende del orden de vecinos) y el **conjunto** de nodos alcanzados por DFS/BFS (la componente conexa). En un examen, si te piden “el” árbol BFS/DFS de un grafo sin especificar el orden de los vecinos, cualquier árbol válido con las capas/alcanzabilidad correctas debe aceptarse — pero conviene aclarar explícitamente qué orden de vecinos se usó.

Trampa 5 — en DFS no dirigido, tratar una back edge repetida (hacia el padre inmediato) como ciclo. La arista que conecta a un nodo con su propio padre en el árbol DFS **no** es un ciclo — es la misma

arista no dirigida recorrida “en el sentido inverso” (porque en un grafo no dirigido, la arista (u,v) aparece en la lista de adyacencia de ambos extremos). Por eso el chequeo de ciclo excluye explícitamente el caso $v=\text{padre}[u]$ (ver pseudocódigo de la sección 3.1/3.3): solo cuenta como ciclo si v es un ancestro **distinto** del padre inmediato.

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	Espacio de una lista de adyacencia vs. matriz para un grafo de n nodos, m aristas	Lista: $O(n+m)$. Matriz: $O(n^2)$, siempre, sin importar m
2	¿Cuál estructura de datos convierte la exploración genérica en BFS?	Una cola FIFO para los nodos descubiertos-no-procesados
3	¿Cuál estructura de datos convierte la exploración genérica en DFS?	Una pila LIFO (o recursión)
4	¿Por qué BFS encuentra caminos más cortos (no pesados)?	Porque ninguna arista de G puede conectar capas que difieran en más de 1 (Prop. 3.4) — todo camino debe atravesar las capas una por una, en orden
5	¿Qué tipo de arista NO puede aparecer en un DFS de grafo no dirigido?	Aristas cruzadas (cross edges) entre ramas sin relación de ancestro — toda arista no-árbol es una back edge (Prop. 3.7)
6	¿Cómo se detecta un ciclo con DFS en no dirigido?	Si al explorar (u,v) , v ya está <code>enPila</code> (gris) y $v \neq \text{padre}[u] \rightarrow$ back edge $\rightarrow$ ciclo
7	Complejidad de BFS y de DFS (con lista de adyacencia)	Ambos $O(n+m)$
8	¿Cómo se calculan TODAS las componentes conexas de un grafo posiblemente desconectado?	Bucle exterior: <code>for s=1..n: if not descubierto[s] then BFS(s)</code> — cada llamada nueva es una componente
9	Teorema de bipartición	G bipartito $\Leftrightarrow G$ no tiene ciclos de longitud impar
10	¿Cómo se implementa el 2-coloreo de bipartición?	BFS/DFS asignando <code>color[v] = nivel[v] mod 2</code> ; conflicto si una arista conecta dos nodos del mismo color
11	¿Cómo se calculan las SCC de un grafo dirigido?	Algoritmo de Kosaraju: DFS en G por orden de finalización, luego DFS en G^{rev} en orden decreciente de esa finalización
12	¿Cuándo existe un orden topológico?	$\Leftrightarrow$ el grafo dirigido es un DAG (sin ciclos dirigidos)
13	Algoritmo y complejidad de orden topológico	Algoritmo de Kahn (grados de entrada + cola de grado 0), $O(n+m)$
14	BFS da un camino con costo total mínimo si las aristas tienen pesos distintos, ¿V o F?	Falso — BFS solo minimiza cantidad de aristas; con pesos hace falta Dijkstra (otro módulo)
15	En DFS iterativo del curso, ¿un nodo puede aparecer más de una vez en la pila?	Sí — se apila una vez por cada vecino que lo descubre; solo se procesa la primera vez que se desapila sin estar explorado

Resumen ultra-rápido (2 minutos)

- **Lista de adyacencia:** $O(n+m)$ espacio, $O(\text{grado}(v))$ listar vecinos — la representación por defecto para BFS/DFS. **Matriz:** $O(n^2)$ espacio siempre, $O(1)$ consultar arista puntual.
- **Exploración genérica (KT 3.2):** conjunto S de descubiertos, agregar mientras haya arista (u,v) con $u \in S$, $v \notin S$. **BFS = cola. DFS = pila/recursión.**
- **BFS:** procesa por capas $L_0, L_1, \dots$; ninguna arista salta más de una capa (Prop. 3.4) $\Rightarrow$ nivel(t) = distancia más corta $s \rightarrow t$ (solo grafos **no pesados**). Complejidad **$O(n+m)$** .
- **DFS:** en no dirigido, toda arista no-árbol es una **back edge** (hacia un ancestro, nunca cruzada — Prop. 3.7). Back edge $\Leftrightarrow$ ciclo. Complejidad **$O(n+m)$** ; en la versión iterativa del curso, espacio auxiliar de la pila hasta $O(m)$ (no $O(n)$), porque un nodo puede apilarse varias veces.
- **Componentes conexas:** BFS/DFS desde s descubre exactamente la componente de s (Prop. 3.8: componentes idénticas o disjuntas, nunca parcialmente solapadas). Para TODO el grafo: bucle exterior `for s: if not descubierto[s] then BFS(s)`.
- **Bipartición:** G bipartito $\Leftrightarrow$ sin ciclos de longitud impar. 2-coloreo por BFS: color = nivel mod 2; conflicto (misma color en ambos extremos de una arista) $\Leftrightarrow$ ciclo impar encontrado.
- **Dirigidos:** SCC vía Kosaraju (DFS en G por finalización + DFS en G^{rev} en orden inverso). Orden topológico existe $\Leftrightarrow$ DAG; algoritmo de Kahn con grados de entrada, $O(n+m)$.
- **Las 5 trampas de examen:** (1) BFS $\neq$ caminos mínimos con pesos (eso es Dijkstra); (2) marcar visitado en el momento correcto o se rompe la complejidad/termina mal; (3) nunca asumir grafo conexo sin el bucle exterior; (4) el árbol BFS/DFS no es único, pero capas y alcanzabilidad sí; (5) la arista hacia el padre inmediato en DFS no dirigido no es un ciclo.